

6.5830/1 Lecture 18: Eventual Consistency

4/16/2026

Tianyu Li

litianyu@mit.edu

Administrivia

- Lab 3 due today
 - Writeup 4/21
- Project Meetings this week + midterm reports (4/19)
- Quiz 2 next week
 - Same rules as quiz 1

Amazon Operational DB Desiderata

- "Always Available" shopping cart
 - Should not go down even if a datacenter fails
 - No centralized point of failure
 - Some "incorrectness" is ok
 - No complex queries
- Very low latency
 - Lots of orders being processed
 - Many lookups required to render a page
- Very large scale
 - Lots of clients, many stores, Black Friday, oh my!

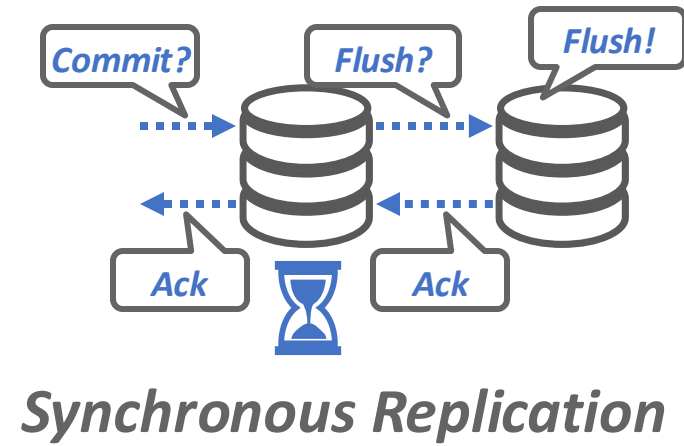
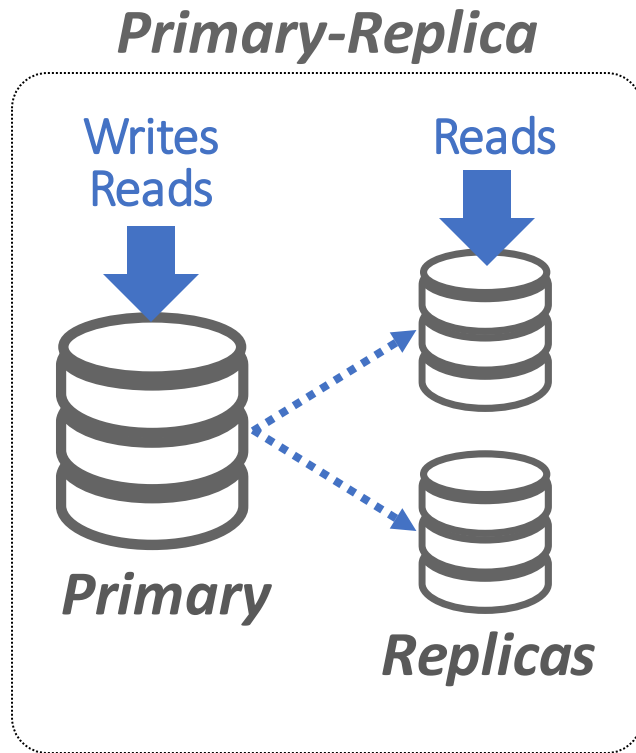


Recap: Distributed DB

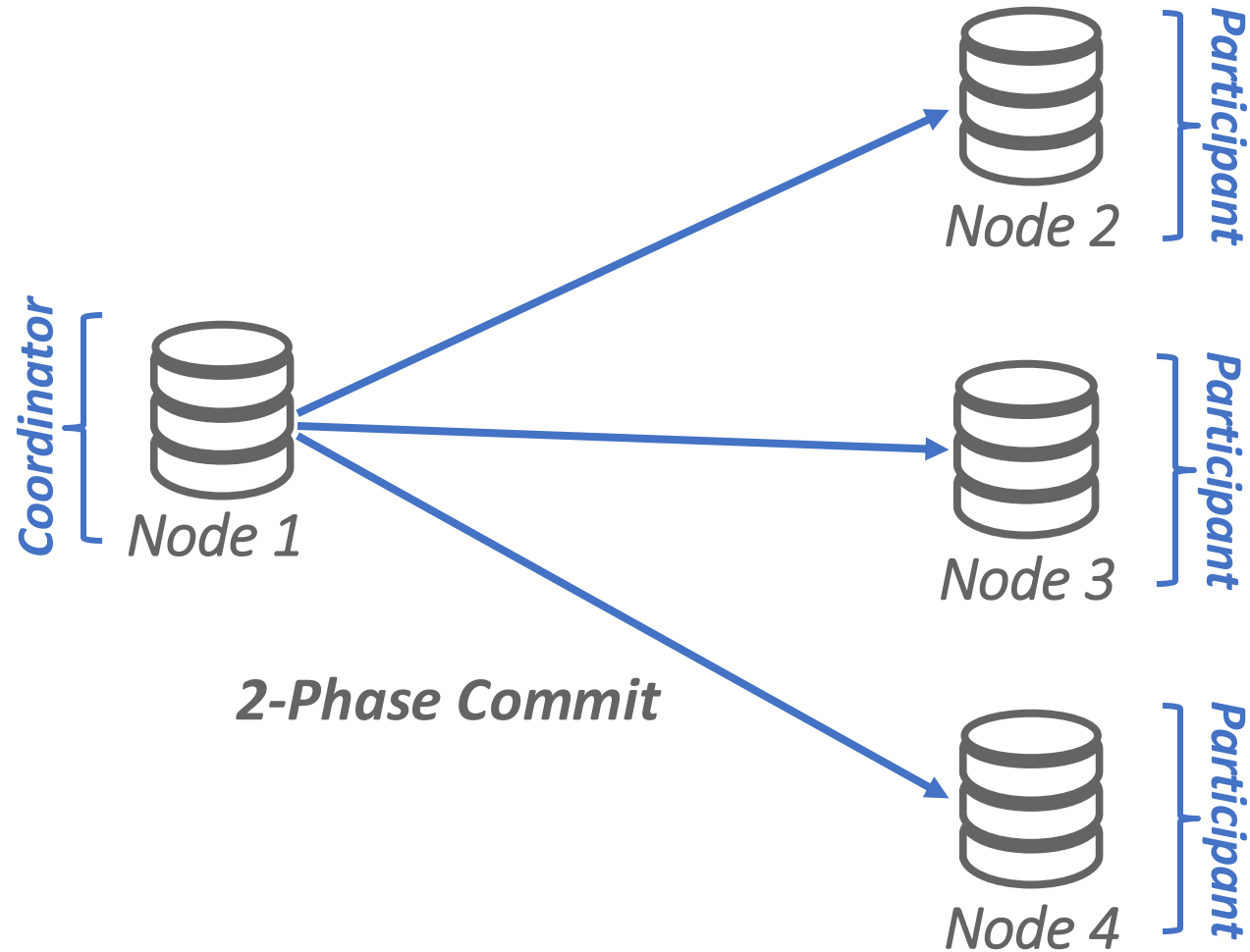
- Sharding + Replication to provide the same guarantee as a single node DBMS
 - i.e., ACID Transactions
- Many design choices
 - Replica Configuration
 - Propagation Scheme
 - Propagation Timing
 - Update Method



Traditional DBMS: Design Choices



Traditional DBMS: Design Choices



Problem

- Recall: under 2PC, failure of participant/coordinator can result in blocked progress until recovery
 - Or at least, until a large timeout in good cases
- User clicks “add to cart”
 - Button spins
 - User loses interest and clicks exit
 - ACID guarantee: transaction will NOT commit
 - Therefore, user no pay you



More Problems

- Synchronous replication is safe and improves read performance
- Writes may actually become slower
 - Must write at multiple sites and wait for them to confirm
 - Database will block if replicas do not respond
- User clicks “add to cart”
 - Button spins
 - User loses interest and clicks exit
 - ACID guarantee: transaction will NOT commit
 - Therefore, user no pay you

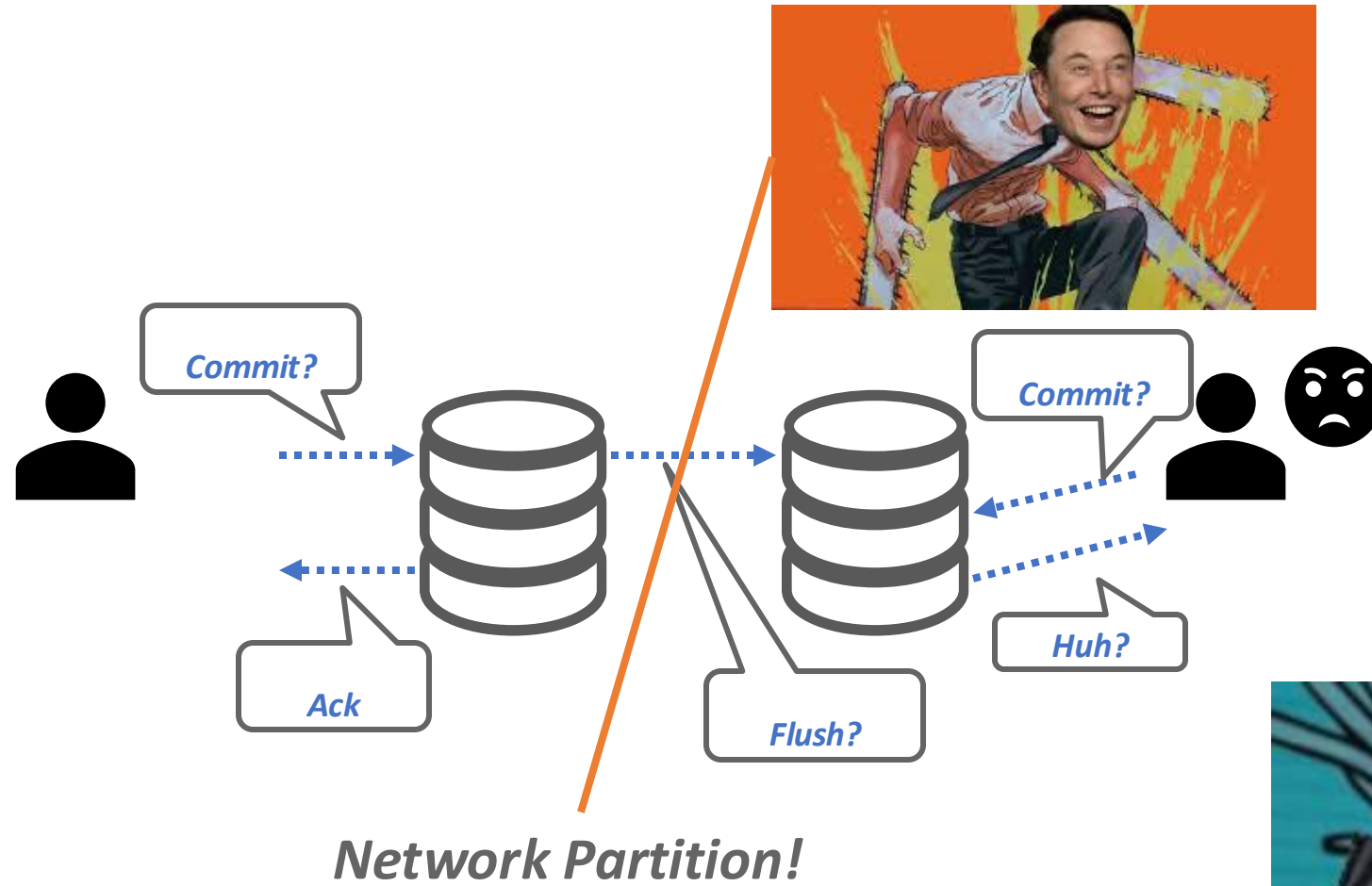


Idea

- Don't support transactions
 - Why would I atomically update my shopping cart?
 - ACID is for nerds
 - No 2PC
- Still need replicas
 - So your shopping cart can survive wars
 - But: do NOT wait for replication acks so it will be FAST!



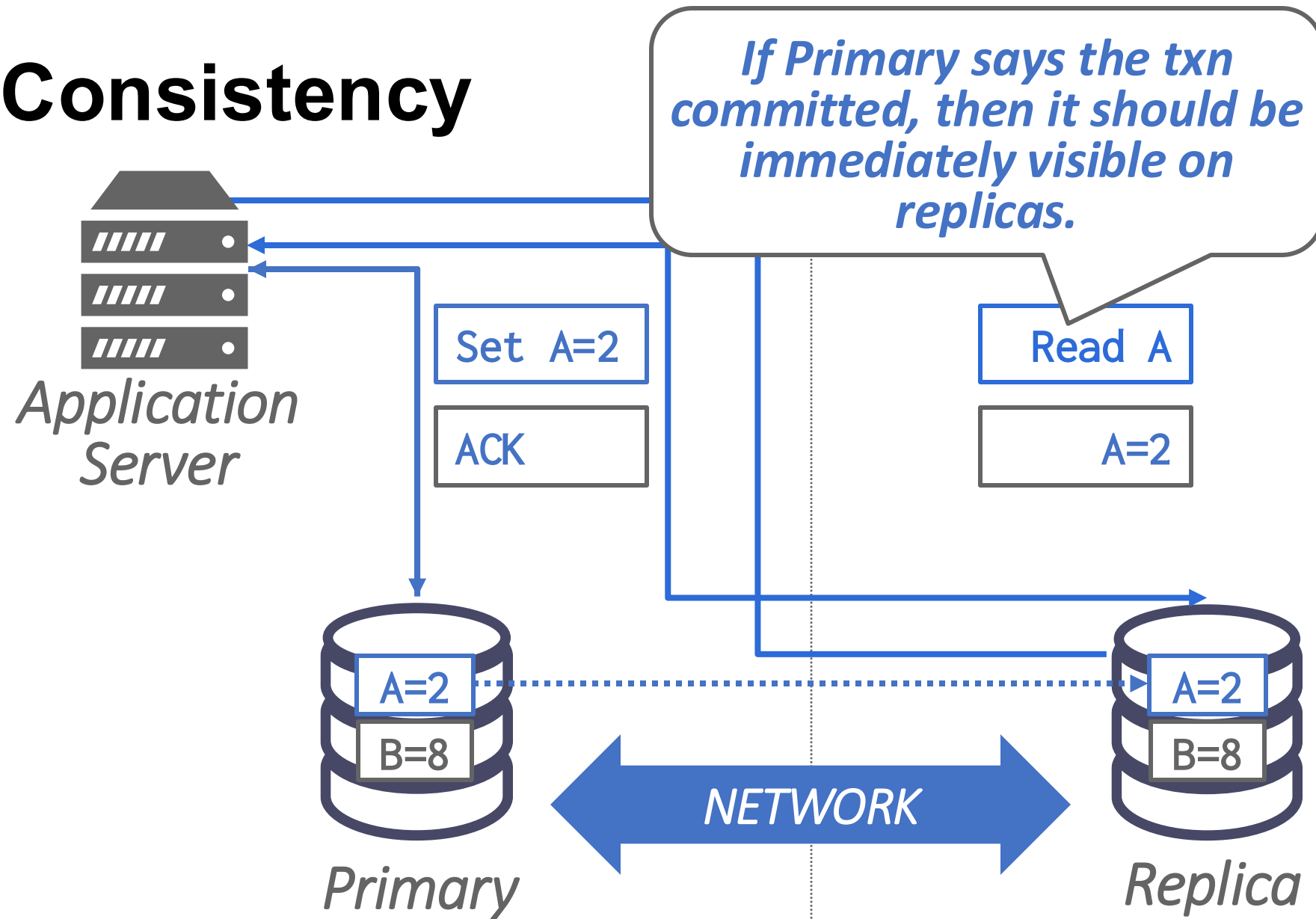
Even More Problems



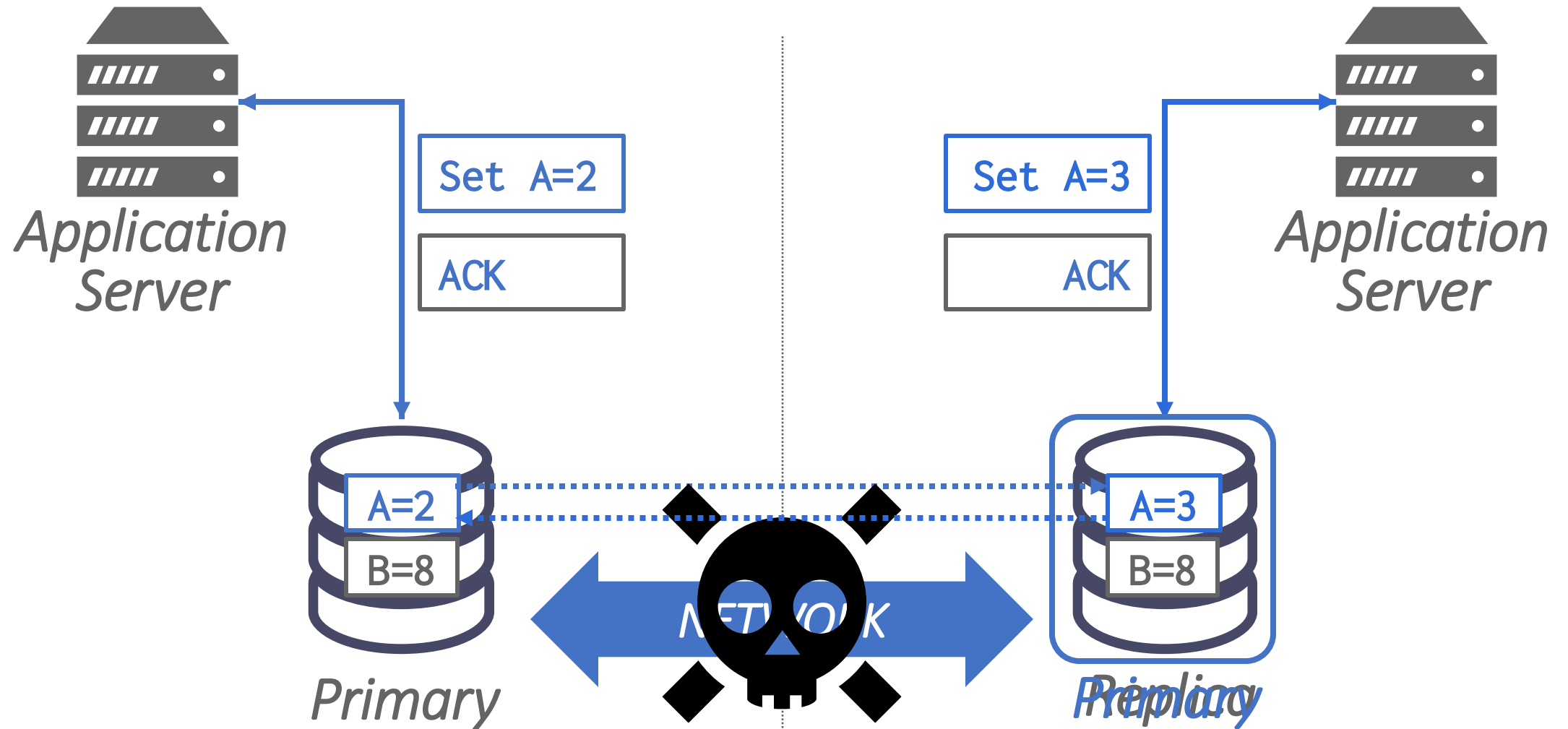
CAP THEOREM

- Proposed by Eric Brewer in the late 1990s that is impossible for a distributed database to always be:
 - Consistent
 - Always Available
 - Partition Tolerant
- Later proved under asynchronous network assumption by Gilbert & Lynch of MIT
- More formally, partition tolerance is an operating assumption. CAP theorem means we must choose between C and A

Consistency



Availability



What to do in a Partition?

Traditional DBMS

- **Choice #1: Halt the System**

- Stop accepting updates in any partition that does not have a majority of the nodes.

- **Choice #2: Allow Split, Reconcile Changes**

- Allow each side of partition to keep accepting updates.
- Writes may appear lost, state may diverge
- Handle reconciliation upon reconnection

Eventually Consistent Systems

Amazon Dynamo

- Key / Value store, no cross-key atomicity
 - All keys and values are arbitrary byte arrays
 - Simple get/put interface
- Partitioned and replicated by key
- Highly available, but only eventually consistent in the face of network partitions and failures
 - Many other systems like this, often called “NoSQL”

Dynamo: Ideas

- Consistent Hashing
- Gossip based membership
- Sloppy Quorums + Hinted Hand-off
- Vector Clocks
- Anti-Entropy

Recall:

Partitioning Key

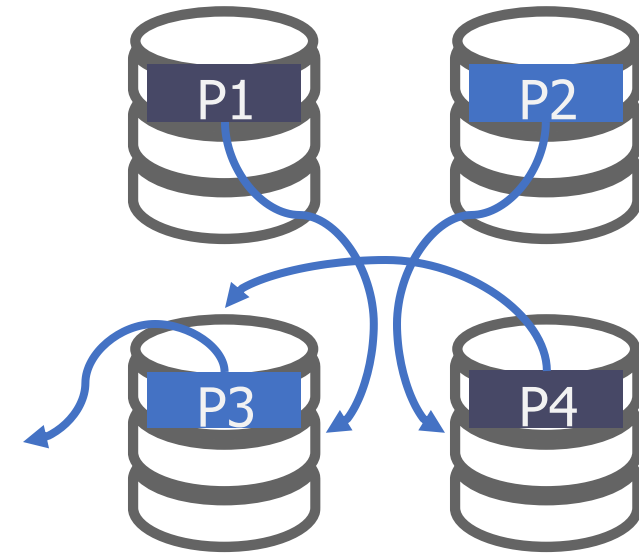
Table

101	a	XXX	2025-11-29	$\text{hash}(a)\%3 = P2$
102	b	XXY	2025-11-28	$\text{hash}(b)\%3 = P3$
103	c	XYZ	2025-11-29	$\text{hash}(c)\%3 = P3$
104	d	XYX	2025-11-27	$\text{hash}(d)\%3 = P2$
105	e	XYY	2025-11-29	$\text{hash}(e)\%3 = P3$

Ideal Query:

```
SELECT * FROM table  
WHERE partitionKey = ?
```

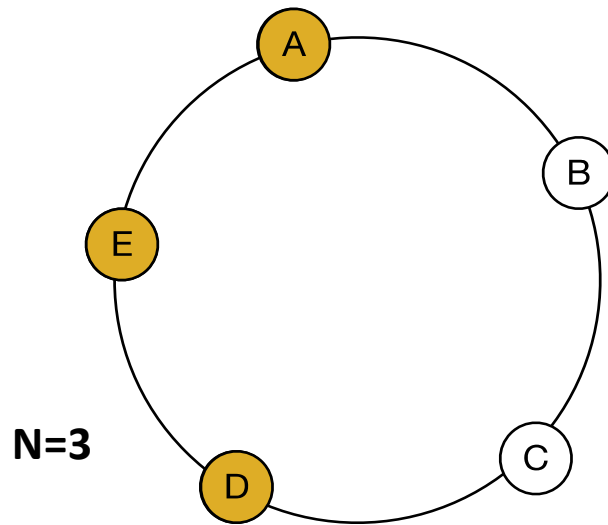
Partitions



Consistent Hashing

- All data replicated on N nodes
- Each node has an address on a “ring” representing space of hash values from say, $0 \rightarrow 2^{128}$
- Data stored on ring as well

*“Overlay network”:
Nodes are not actually in a physical ring, but are just machines on the Internet*

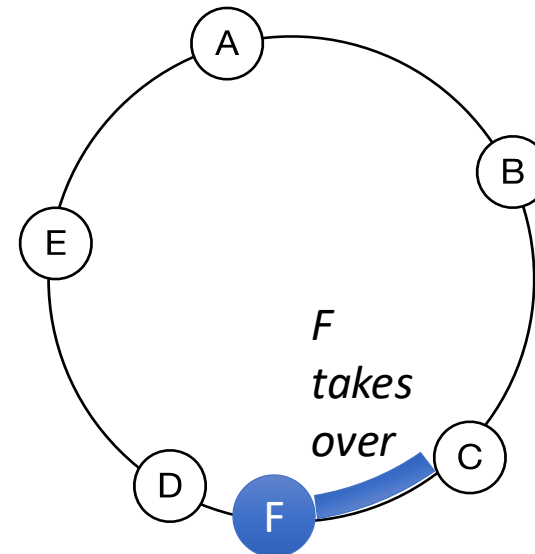


Each node occupies one (or multiple) random locations on ring

k A key hashed to location **k** is stored on *N successors* in ring

Consistent Hashing

- Data and nodes mapped to ring
- Data assigned to nearest successor(s)
- When a node joins, it takes over only keys in range it joins
- No need to rehash all values!



Data Skew

- In practice, can lead to load imbalance if not careful
- Fix: “virtual” nodes
 - Each physical nodes has multiple (100s of) identities on the ring
 - Statistically reduce likelihood of hot spots
 - Must take extra care when encountering virtual nodes belonging to the same physical nodes

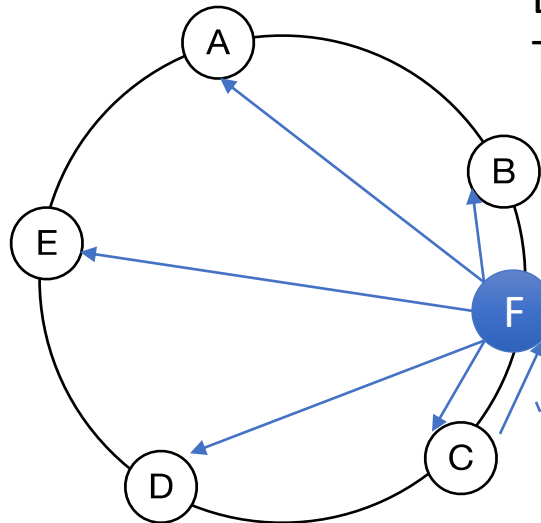
Joining the Ring

- Administrators explicitly add / remove nodes
- When a node joins, it contacts a list of “seed nodes”
 - Other nodes periodically “gossip” to learn about ring structure
- When a node i learns about new node j , i sends j any keys j is responsible for

External discovery service

Seed node list

Seed nodes are nodes clients and other nodes can ask for current mapping



Each node has mapping of all other nodes. This is small, even for thousands of nodes

Node	Loc
A	0
B	X
C	Y
F	W

B Table

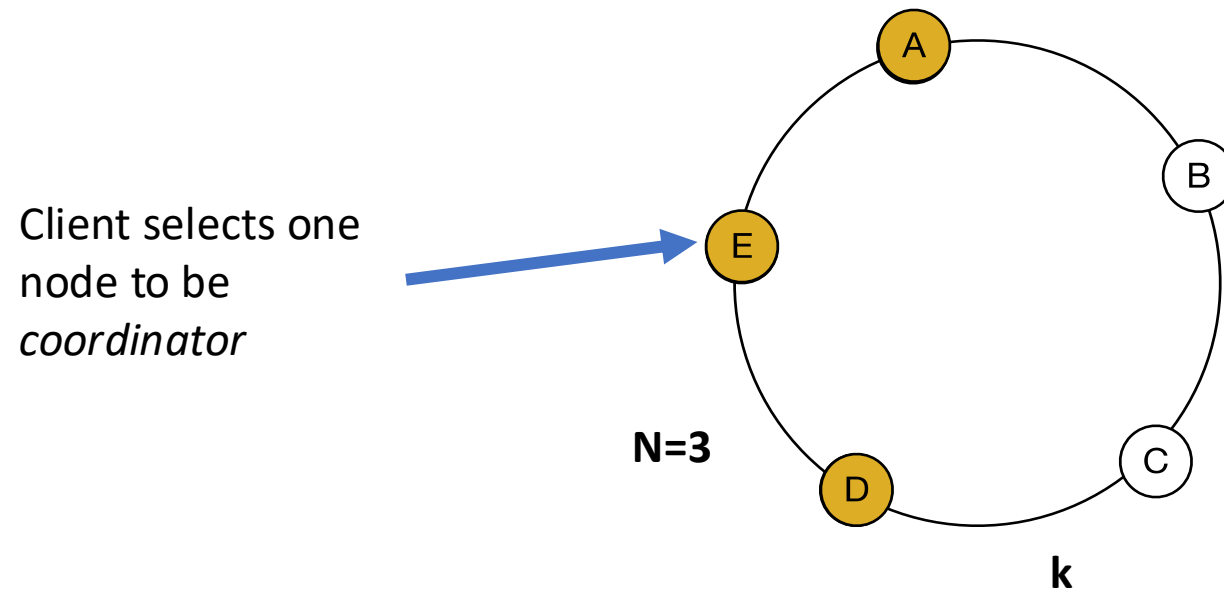
Node	Loc
A	0
B	X
C	Y
F	W

C Table

Keys in range $[X, W]$

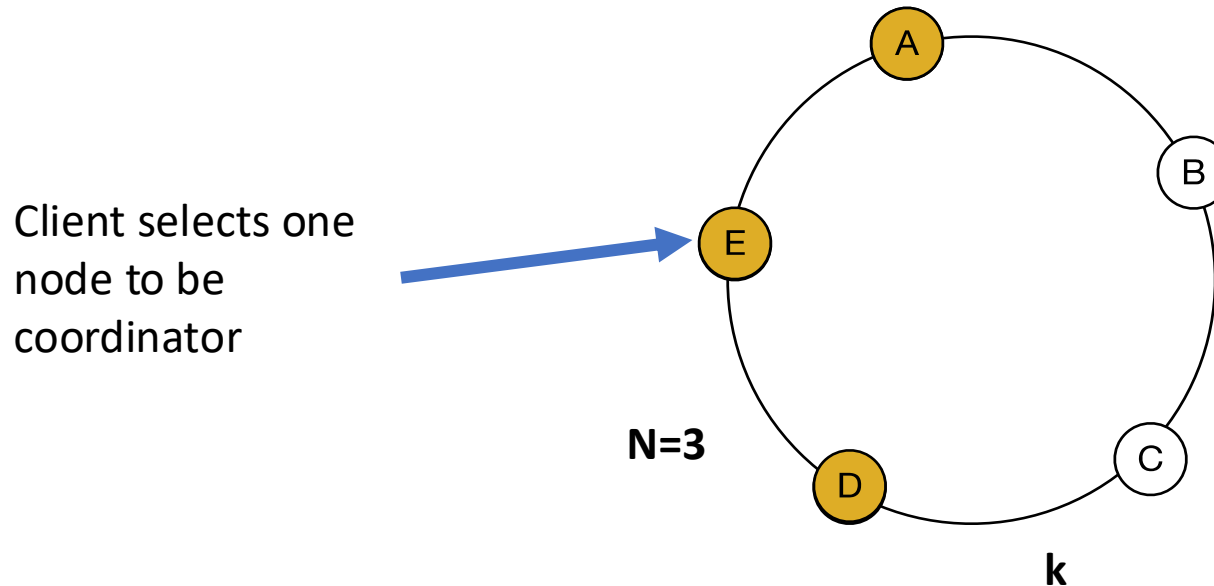
Handling Reads

- Each item is replicated on N nodes
- To read: hash key, send request to one replica
 - Client either uses Amazon front end or reads mapping table from seeds



Handling Writes

- Route as in reads
- Back to our availability conundrum
 - Do we write all replicas? What if one has failed / isn't available?
 - Do we write just one replica? How do we ensure that our read will be visible to other nodes?



Idea

- “Quorum Writes”
- $R + W > N$
 - N = number of replicas of each data item
 - R = number of replicas each read must be heard from
 - W = number of replicas each write must be sent to
- E.g., $R = 2, W = 2, N = 3$

<u>R1</u>	<u>R2</u>	<u>R3</u>
-----------	-----------	-----------

v1	v1	v1
----	----	----

v2	v1	v2
----	----	----

 ← write to 2 out of 3

Any read of 2 will see v2!



Problem

- Recall: CAP theorem
 - Strict, overlapping quorums indeed ensure consistency, therefore, it must not be highly available
- During a partition, cannot do some operations without majority
 - In worst case, half of the system is down
- Half of the network is still a lot of customers



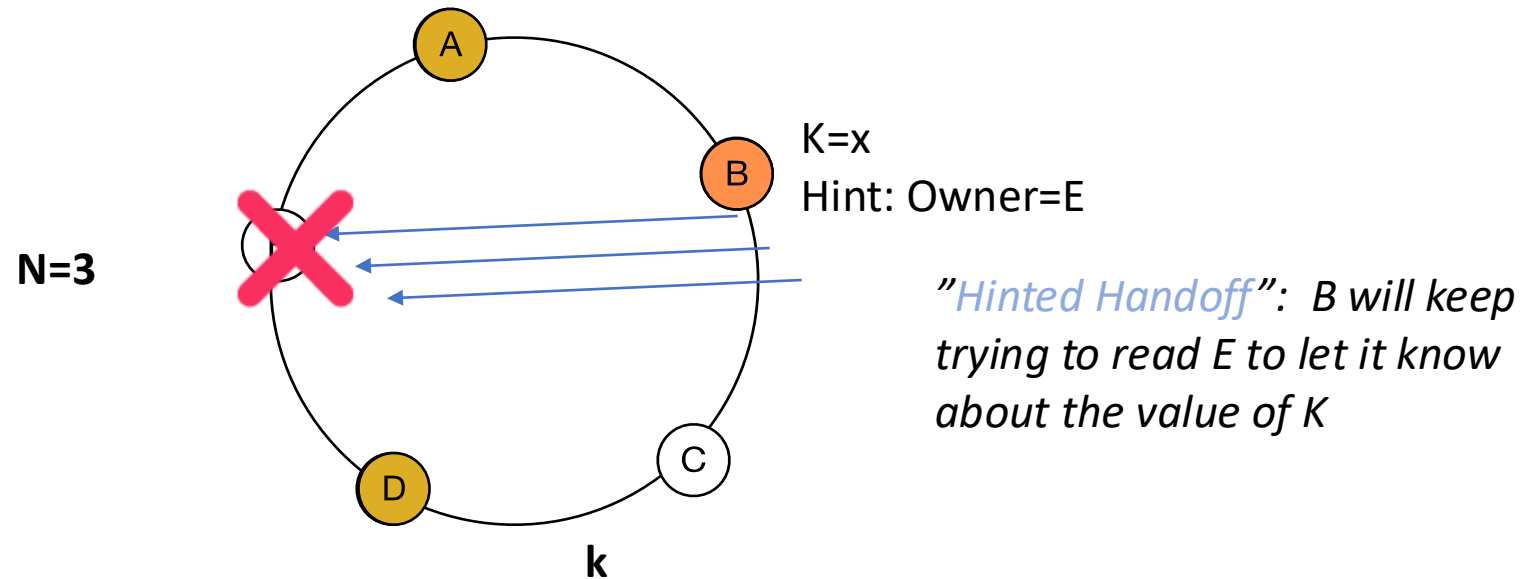
More Ideas

- Why don't we just have non-overlapping quorums?
- Problem: this will always allow inconsistencies, regardless of operating conditions
 - A drone will (hopefully) not hit your data center every day
- Want: only allow inconsistencies when failures occur as a fallback



Sloppy Quorum & Hinted Handoff

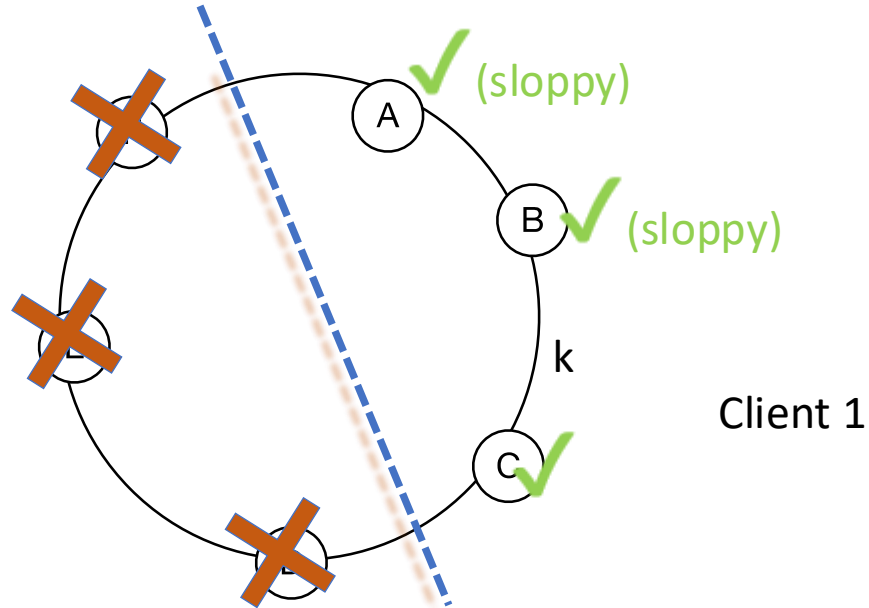
- If fewer than N writes succeed, continue around ring, past successors



***2 out of 3 writes succeed
Continue around ring, write to B***

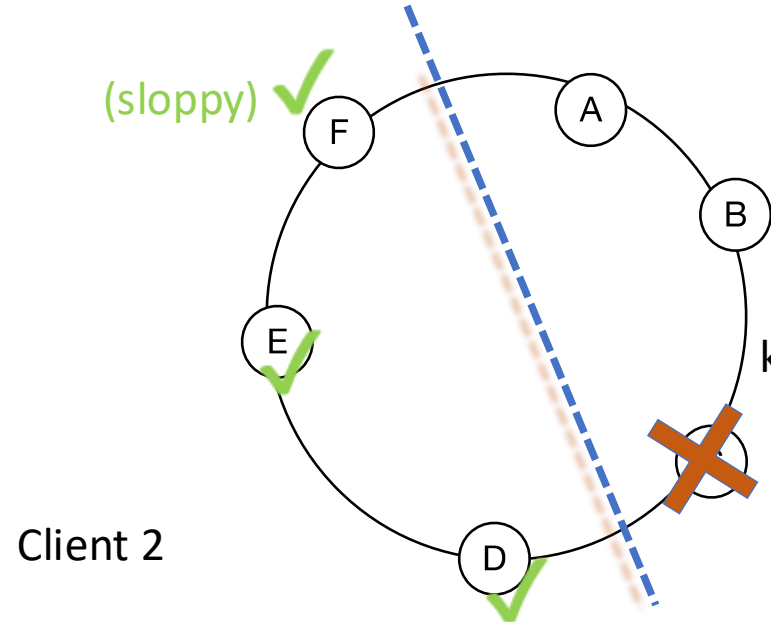
Sloppy Quorum → Divergence

- If network is partitioned, hinted handoff can lead to divergent replicas
- E.g., suppose $N=3$, $W=2$, $R=2$, Partitioned



Sloppy Quorum → Divergence

- If network is partitioned, hinted handoff can lead to divergent replicas
- E.g., suppose $N=3$, $W=2$, $R=2$, Partitioned



Two different
versions of key
 k , k_1 and k_2
now exist

Obvious Idea

- Why don't we just read the latest value?
 - Last writer wins, right?
- Well, first, you would silently lose data
 - That is, if you can even define "latest"



Obvious, you say?

- Leslie Lamport observes that the definition of “time” is unclear in a distributed system
 - Machine clock drifts due to many reasons
 - Communication latency makes it hard to synchronize time reliably
- He then attempted to explain this with the theory of relativity and thought it would be obvious
 - Perhaps not surprising, he is the person who thought it was obvious that a fictional story of an ancient Greek island city state is in fact a consensus algorithm



Time

- Philosophically, time entails the possibility of causality
 - If A happens after B, B may have caused A
- This “happens-before” relation can be established in a distributed system without the need for a synchronized clock
 - If **a** and **b** happened on the same machine, easy to order
 - If **a** is the sending of a message, and **b** is the receipt of that message, then a happens before **b**
 - Transitivity: if **a** happens before **b** and **b** happens before **c**, a happens before **c**

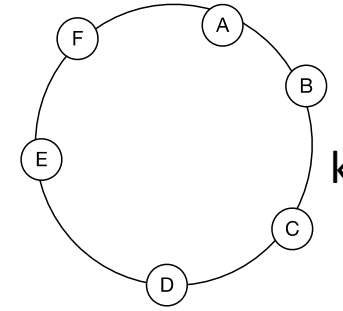
The Lamport Clock

- An algorithm that establishes logical time that has most nice properties of time that we like
 - Each process maintains a local counter and piggybacks this counter to messages it send
 - Receiving process updates its counter to be greater than both its current value and the received message
- Problem: this clock allows for concurrent events
 - If neither **a** happens before **b** nor **b** happens before **a**
 - **a** happens before **b** implies $\text{timestamp}(a) < \text{timestamp}(b)$, but the converse is NOT true

Vector Clocks

- Lamport claims that this problem is easy to solve by appending unique ID to process time and imposing an arbitrary total order amongst the IDs
 - Unsurprisingly, it is not easy, but, he is correct that this is the solution
- Practical systems use vector clocks
 - Instead of a single counter, maintain one for each process
 - Similar rules to Lamport clock, but receiving increment counter element-wise
 - Now, happens-before works in both directions
 - Critically, this allows us to *identify* concurrency

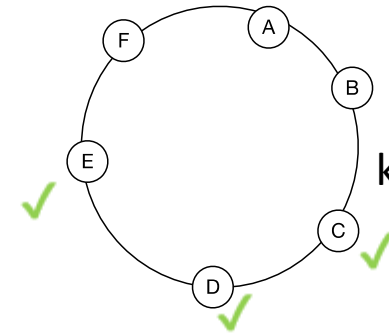
Vector Clocks



- Each node keeps a monotonic version counter that increments for every write it *coordinates*
- Each data item has a *clock*, consisting of a list of the most recent version it includes from each coordinator

A	B	C	D	E	F

Vector Clocks



- Each node keeps a monotonic version counter that increments for every write it *coordinates*
- Each data item has a *clock*, consisting of a list of the most recent version it includes from each coordinator

Client 1

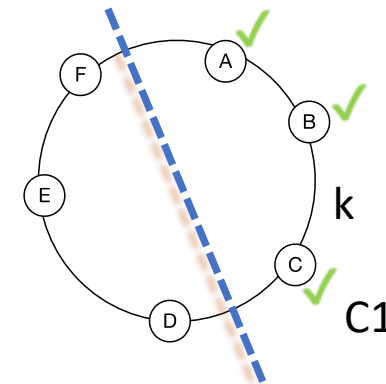
Create $k \rightarrow C$

C writes $[C,1]$ to C, D, E

A	B	C	D	E	F
		1 [C,1]	1 [C,1]	1 [C,1]	

[C,1]: Contains
first version from
C as coordinator

Vector Clocks



- Each node keeps a monotonic version counter that increments for every write it *coordinates*
- Each data item has a *clock*, consisting of a list of the most recent version it includes from each coordinator

Client 1

Create $k \rightarrow C$

C writes $[C,1]$ to C, D, E

Client 1

Read $k \rightarrow C$

C reads C, D, E

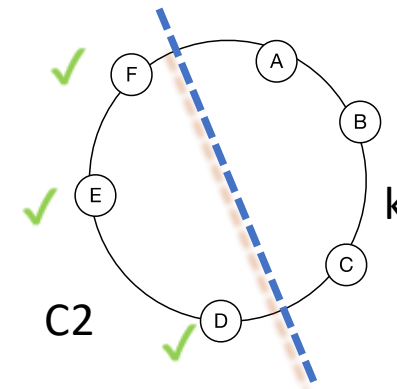
C returns $[C,1]$

Write $k [C, 1] \rightarrow C$

C writes $[C,2] \rightarrow C, A, B$

A	B	C	D	E	F
		1 [C,1]	1 [C,1]	1 [C,1]	
2 [C,2]	2 [C,2]	2 [C,2]			

Vector Clocks



- Each node keeps a monotonic version counter that increments for every write it *coordinates*
- Each data item has a *clock*, consisting of a list of the most recent version it includes from each coordinator

Client 2

Read $k \rightarrow D$

D reads D,E,F

D returns [C,1]

Write k [C, 1] $\rightarrow D$

D writes [C,1][D,1]
to D, E, F

A	B	C	D	E	F
		1 [C,1]	1 [C,1]	1 [C,1]	
2 [C,2]	2 [C,2]	2 [C,2]			
			3 [C,1][D,1]	3 [C,1][D,1]	3 [C,1][D,1]

Incomparable
(can't totally order)

Read Repair

- Possible for a client to read 2 incomparable versions
- Need *reconciliation*; options:
 - Latest writer wins
 - Application specific reconciliation (e.g., shopping cart union)
 - You will pay for ALL of the items in your shopping cart
- After reconciliation, perform *write back*, so replicas know about new state



Study Break

Suppose there are three replicas, $R1$, $R2$, and $R3$. Three writes are performed to key K , resulting in three version clocks

$V1 = \langle R1 : 0, R2 : 3, R3 : 2 \rangle$

$V2 = \langle R1 : 1, R2 : 3, R3 : 2 \rangle$

$V3 = \langle R1 : 0, R2 : 0, R3 : 3 \rangle$

Which of the following are true statements?

Study Break

$V1 = \langle R1 : 0, R2 : 3, R3 : 2 \rangle$

V2 was coordinated by R1, saw same versions as V1

$V2 = \langle R1 : 1, R2 : 3, R3 : 2 \rangle$

$V3 = \langle R1 : 0, R2 : 0, R3 : 3 \rangle$

V3 was coordinated by R3, did not see R2 1, 2, or 3, and happened concurrently with V2

1. The writer that produced V_1 observed V_2 .
2. The writer that produced V_2 observed V_1 .
3. The writer that produced V_3 observed V_1 .

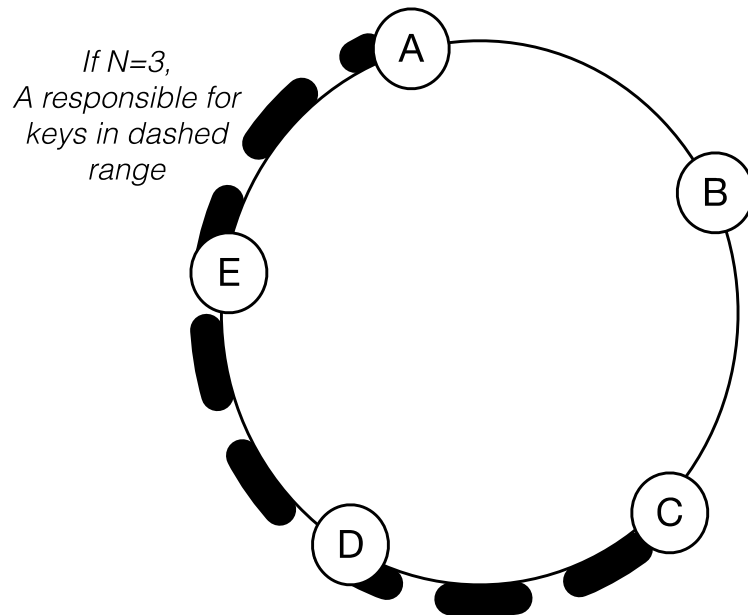
✗

✓

✗

Anti-entropy

- Once a partition heals, or a node recovers, need a way to patch up
- Could rely on gossip & hinted handoff
- Dynamo also compares nodes responsible for each key range
 - Comparison done via hashing, using a technique called *Merkle trees*



Here, for EA range, B and C
are also responsible

Merkle Trees

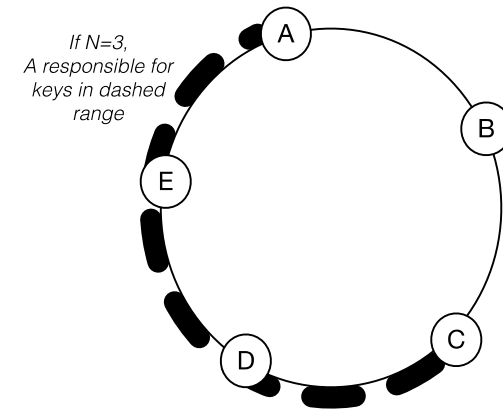
Suppose EA range has keys u, v, w, x, y, z , A and B are comparing

A

$$H(u, v, w, x, y, z) = h1$$

B

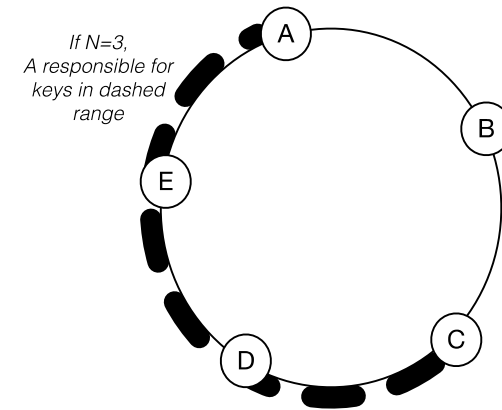
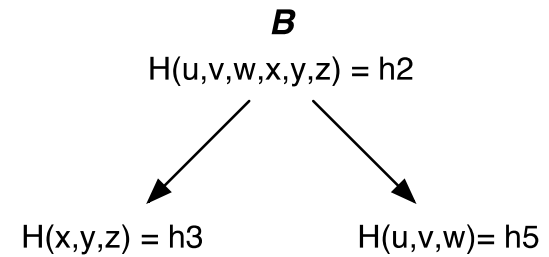
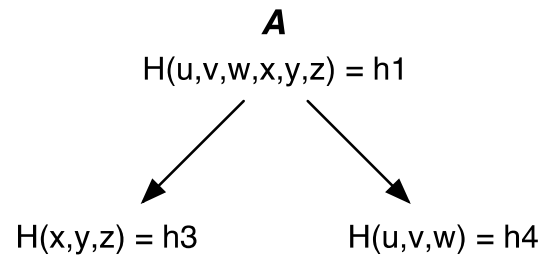
$$H(u, v, w, x, y, z) = h2$$



Here, for EA range, B and C are also responsible

Merkle Trees

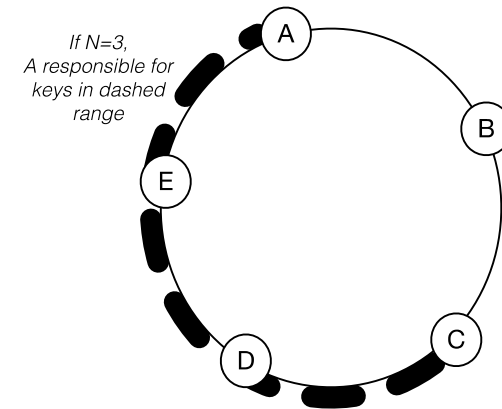
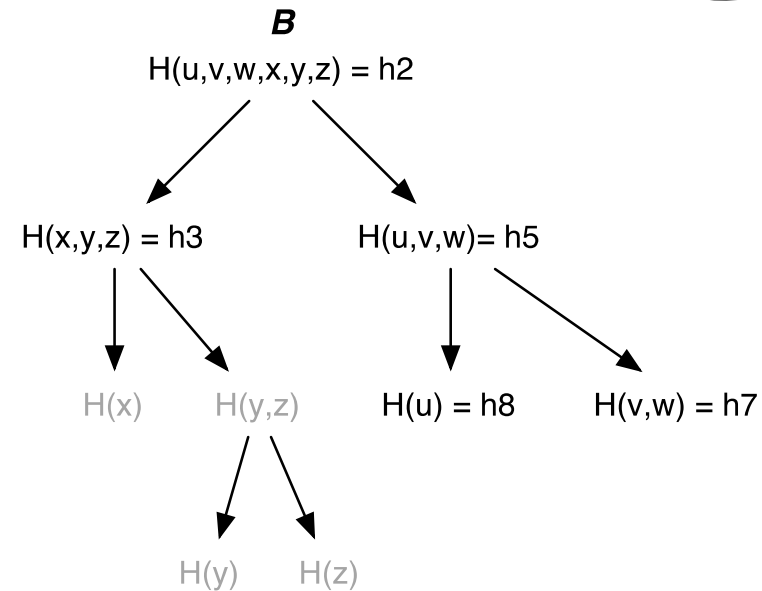
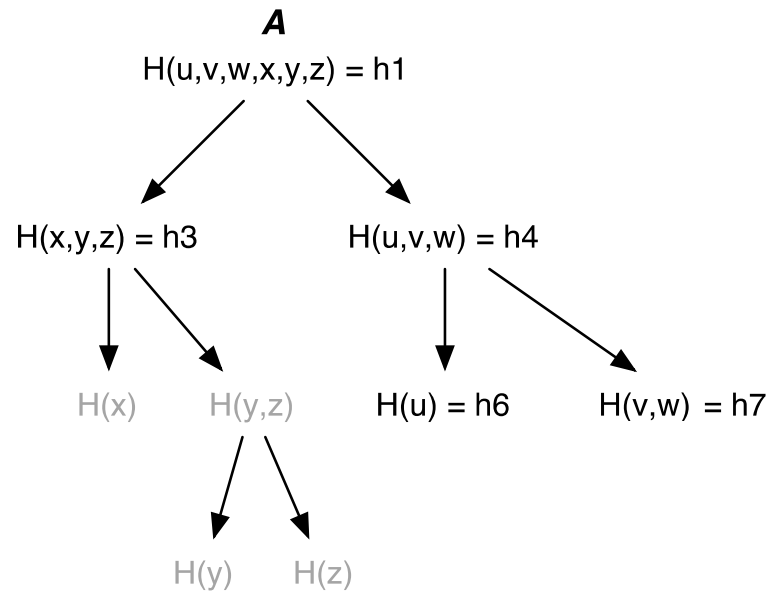
Suppose EA range has keys u, v, w, x, y, z , A and B are comparing



Here, for EA range, B and C are also responsible

Merkle Trees

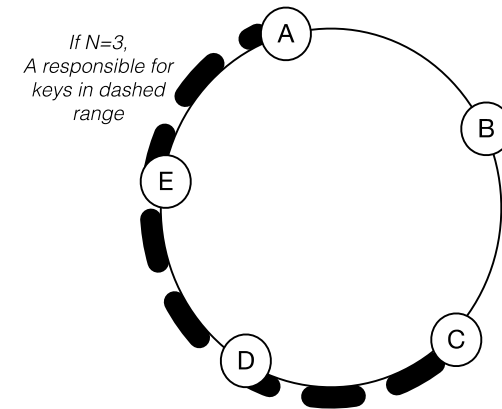
Suppose EA range has keys u, v, w, x, y, z , A and B are comparing



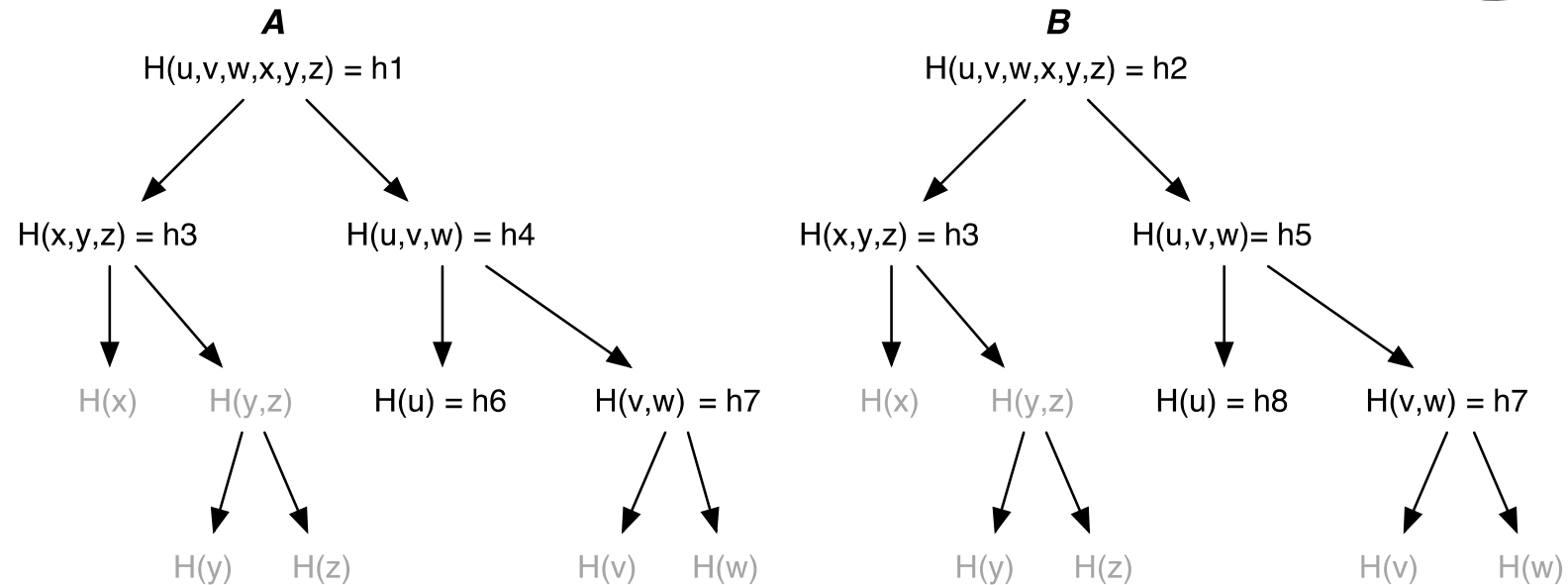
Here, for EA range, B and C are also responsible

Merkle Trees

Suppose EA range has keys u, v, w, x, y, z , A and B are comparing

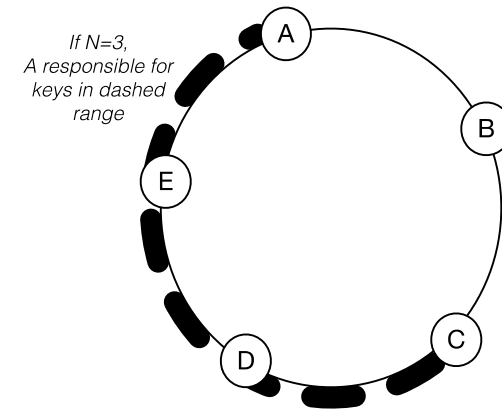
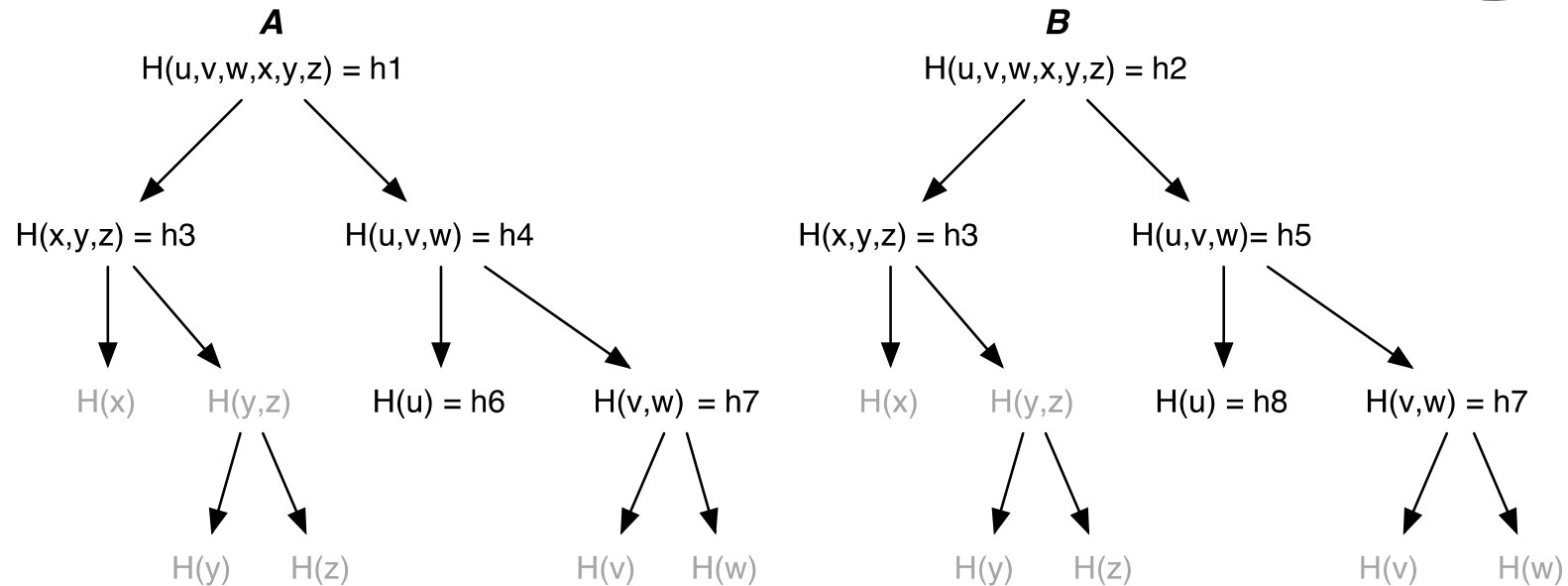


Here, for EA range, B and C are also responsible



Merkle Trees

Suppose EA range has keys u, v, w, x, y, z , A and B are comparing



Here, for EA range, B and C are also responsible

This whole tree is as big as data, but only need to exchange parts of it that are different, i.e., no need to send light gray nodes in diagram, since parent hashes are all equal

Summary

Problem	Technique	Purpose
Partitioning	Consistent hashing	Incremental scalability



Summary

Problem	Technique	Purpose
Partitioning	Consistent hashing	Incremental scalability
Highly available for writes	Vector clocks with read repair	Version size decoupled from update rate



Summary

Problem	Technique	Purpose
Partitioning	Consistent hashing	Incremental scalability
Highly available for writes	Vector clocks with read repair	Version size decoupled from update rate
Handle temporary failures	Sloppy quorum and hinted handoff	HA with some durability



Summary

Problem	Technique	Purpose
Partitioning	Consistent hashing	Incremental scalability
Highly available for writes	Vector clocks with read repair	Version size decoupled from update rate
Handle temporary failures	Sloppy quorum and hinted handoff	HA with some durability
Recovery from permanent failures	Anti-entropy	Sync replicas w/ Merkle Trees



Summary

Problem	Technique	Purpose
Partitioning	Consistent hashing	Incremental scalability
Highly available for writes	Vector clocks with read repair	Version size decoupled from update rate
Handle temporary failures	Sloppy quorum and hinted handoff	HA with some durability
Recovery from permanent failures	Anti-entropy	Sync replicas w/ Merkle Trees
Membership / failure detection	Gossip based membership	Symmetry and no centralized coordination

